
Threads

Sistemas Operacionais - Laboratório 5

Curso: Engenharia de Telecomunicações
Professor: Rogerio Pereira Junior

Aluno:
Victor E. de L. Guerra

31/03/2026

Sumário

1	Objetivos	2
2	Experimento 1: Criando processos com fork() e analisando com ps	2
3	Experimento 2: fork() + execve() e análise com ps	2
4	Experimento 3: Criando múltiplos processos e visualizando com ps- tree + grep	3
5	Experimento 4: Quantidade de Processos e Saídas na Tela	4
5.1	Código 1	4
5.2	Código 2	4
6	Experimento 5: Sincronização com wait()	5
6.1	Questões para o Relatório	6
7	Experimento 6: A Primeira Thread (Simples)	6
8	Experimento 7: O Contador de Threads	7
8.1	Respostas	7
9	Experimento 8: Concorrência e Race Condition	8
10	Experimento 9: Monitoramento de Longa Duração	9
11	Experimento 10: Simulação de Servidor Web	9
12	Desafio Final: Análise Teórica de Modelos de Threads	9
12.1	A) Diagramas de Execução	9
12.2	B) Durações Aproximadas	10
12.3	C) Saída do Programa	10

1 Objetivos

- Compreender a criação e o ciclo de vida de Threads em C (Pthreads)
- Analisar o escalonamento, prioridades e estados de processos via Shell.
- Observar problemas de concorrência (Race Conditions).
- Praticar o controle de jobs (Foreground/Background) e sinais de terminação.

2 Experimento 1: Criando processos com fork() e analisando com ps

Este experimento tem como objetivo demonstrar a criação de processos em sistemas Unix utilizando a função `fork()`. Ao executar o programa, são gerados dois processos: o processo pai e o processo filho, cada um com seu identificador (PID). Em seguida, utilizou-se o comando `ps -f` para listar os processos em execução, permitindo identificar e diferenciar o processo pai do filho com base nos campos PID (Process ID) e PPID (Parent Process ID).

F	UID	PID	PPID	PRI	NI	VSZ	RSS	WCHAN	STAT	TTY	TIME	COMMAND
0	1001	2875	2845	20	0	11764	8572	do_sel	Ss+	pts/0	0:00	bash
0	1001	17510	17432	20	0	11768	8828	do_wai	Ss	pts/1	0:00	/usr/bin/b
0	1001	18163	17432	20	0	13348	9680	do_wai	Ss	pts/2	0:00	/usr/bin/b
0	1001	24680	17510	20	0	2472	884	hrttime	S+	pts/1	0:00	./ex1
1	1001	24681	24680	20	0	2472	96	hrttime	S+	pts/1	0:00	./ex1

Figura 1: Output do terminal pelo comando `ps -gl`

Com base na saída que vemos no print acima, podemos afirmar que o processo pai é o processo com PID **24036** e o processo filho é o processo com PID **24037**. Podemos identificar isso por conta o PPID (Parent PID), confirmando que o processo **24037** é filho do processo **24036**

3 Experimento 2: fork() + execve() e análise com ps

Este experimento tem como objetivo demonstrar a substituição de um processo utilizando a função `execve()`. Após a criação de um processo filho com `fork()`, o processo filho executa o programa `/bin/ls`, substituindo completamente seu código original. Enquanto isso, o processo pai continua sua execução normalmente. O comando `ps -gl` foi utilizado para observar o comportamento dos processos, evidenciando que o processo filho passa a executar um novo programa, mantendo, porém, o mesmo PID.

F	UID	PID	PPID	PRI	NI	VSZ	RSS	WCHAN	STAT	TTY	TIME	COMMAND
0	1001	2875	2845	20	0	11764	8572	do_sel	Ss+	pts/0	0:00	bash
0	1001	17510	17432	20	0	11768	8828	do_wai	Ss	pts/1	0:00	/usr/bin/bash --init-file /usr/share/code/resources/app/out/vs/wo
0	1001	18163	17432	20	0	13348	9680	do_wai	Ss	pts/2	0:00	/usr/bin/bash --init-file /usr/share/code/resources/app/out/vs/wo
0	1001	26124	17510	20	0	2472	920	hrttime	S+	pts/1	0:00	./ex2
0	1001	26125	26124	20	0	0	0	-	Z+	pts/1	0:00	[ls] <defunct>
4	1001	28864	18163	20	0	12696	5672	-	R+	pts/2	0:00	ps -gl

Figura 2: Output do terminal pelo comando `ps -gl`

A partir da saída do comando `ps -gl`, é possível identificar o processo que executou o comando `ls` observando a coluna `COMMAND`. No exemplo apresentado, o processo com PID 26125 aparece como `[ls] <defunct>`, indicando que ele executou o comando `ls`.


```

|      |-- ex3
|      '-- ex3

```

5 Experimento 4: Quantidade de Processos e Saídas na Tela

O objetivo deste experimento é compreender como múltiplas chamadas da função `fork()` impactam a quantidade de processos criados e a saída exibida no terminal.

5.1 Código 1

Sem executar o código, observa-se que há duas chamadas de `fork()`. Como cada chamada duplica os processos existentes, o número total de processos será:

$$2^2 = 4 \text{ processos}$$

Portanto, a mensagem "Oi" será exibida 4 vezes.

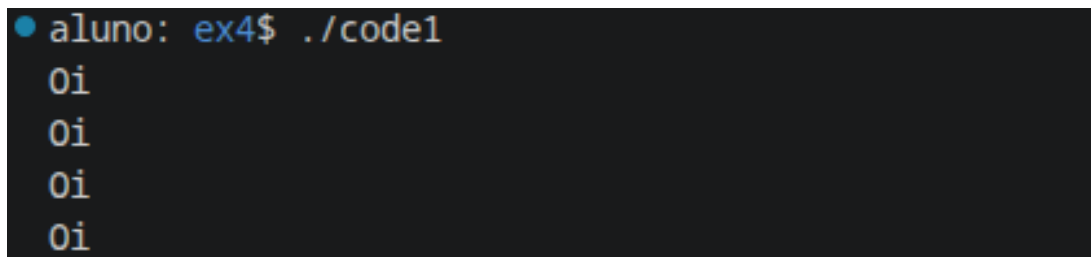
A árvore de processos pode ser representada como:

```

P
|-- P1
|   '-- P3
'-- P2

```

Após a execução, foi possível confirmar que a saída corresponde ao esperado, com a mensagem sendo exibida quatro vezes. Isso ocorre porque cada processo executa o `printf()` de forma independente.



```

aluno: ex4$ ./code1
Oi
Oi
Oi
Oi

```

Figura 4: Execução do Código 1

5.2 Código 2

Neste caso, há três chamadas de `fork()`, resultando em:

$$2^3 = 8 \text{ processos}$$

Assim, a mensagem "Oi" será exibida 8 vezes.

A árvore de processos pode ser representada como:

```

P
|-- P1
|   |-- P3
|   |   |-- P7
|   |   '-- P8
|   '-- P2
'-- P4

```

```

|   '-- P4
|       |-- P9
|       '-- P10
'-- P2
    |-- P5
    |   |-- P11
    |   '-- P12
    '-- P6
        |-- P13
        '-- P14

```

Após a execução, o resultado confirmou a previsão, com a mensagem sendo exibida oito vezes.

O crescimento do número de processos é exponencial, pois cada chamada de `fork()` duplica todos os processos existentes naquele momento. Assim, o total de processos segue a relação 2^n , onde n é o número de chamadas de `fork()`.

```

● aluno: ex4$ ./code2
Oi
Oi
Oi
Oi
Oi
Oi
Oi
Oi
Oi
Oi

```

Figura 5: Execução do Código 2

6 Experimento 5: Sincronização com `wait()`

O objetivo deste experimento é demonstrar a importância da chamada de sistema `wait()` para a sincronização entre processos pai e filho, evitando o surgimento de processos zumbis e garantindo a ordem de execução.

O programa cria um processo filho que executa uma “tarefa pesada” (simulada com `sleep(5)`) e encerra com status 42. O processo pai utiliza `wait(&status)` para aguardar o término do filho antes de prosseguir.

```

[FILHO]: Iniciando tarefa pesada (5 segundos)...
[FILHO]: Tarefa concluída. Vou encerrar agora com status 42.
[PAI]: Meu filho (PID 39902) está trabalhando. Vou aguardar...
[PAI]: Meu filho terminou normalmente com status: 42
[PAI]: Agora que meu filho encerrou, eu também vou. Tchau!
[rerun: b5]

```

Figura 6: Execução do Experimento 5 com `wait()` ativo

```

[PAI]: Meu filho (PID 39959) está trabalhando. Vou aguardar...
[PAI]: Agora que meu filho encerrou, eu também vou. Tchau!
[rerun: b6]

```

Figura 7: Execução do Experimento 5 sem `wait()` (linha comentada)

6.1 Questões para o Relatório

1. No experimento com o `wait()` ativo, quem imprime a última mensagem no terminal? Por quê?

Com o `wait()` ativo, o processo pai imprime a última mensagem no terminal. Isso ocorre porque a chamada `wait(&status)` bloqueia o processo pai até que o filho termine sua execução. Assim, a sequência é: o filho inicia, trabalha por 5 segundos, imprime sua mensagem de conclusão e encerra com `exit(42)`. Somente após isso, o pai retoma a execução, verifica o status do filho e imprime suas mensagens finais.

2. Quando você comentou o `wait()`, o que aconteceu com a ordem das mensagens? O pai esperou o filho?

Sem o `wait()`, o pai **não** esperou o filho. O processo pai continuou sua execução imediatamente após o `fork()`, imprimindo suas mensagens e encerrando antes mesmo do filho completar a tarefa de 5 segundos. Isso fez com que as mensagens do pai aparecessem primeiro, e as mensagens do filho aparecessem depois (ou nem aparecessem, dependendo do momento em que o pai encerrou). A ordem das mensagens ficou invertida em relação ao cenário com `wait()`.

3. O que são as macros `WIFEXITED` e `WEXITSTATUS`? Para que servem?

- `WIFEXITED(status)`: é uma macro que retorna verdadeiro (não-zero) se o processo filho terminou normalmente, ou seja, por meio de uma chamada a `exit()` ou por retorno da função `main()`. Serve para verificar se o filho não foi encerrado por um sinal.
- `WEXITSTATUS(status)`: é uma macro que extrai o código de saída do processo filho (o valor passado para `exit()`). No caso deste experimento, retorna o valor 42, que foi o status de saída definido pelo filho.

4. Explique por que o `wait()` é fundamental para evitar que o sistema fique cheio de processos “zumbis”.

Quando um processo filho termina, ele entra no estado *zombie* (zumbi): o processo já encerrou sua execução, mas sua entrada na tabela de processos ainda existe, aguardando que o pai colete seu status de término. A chamada `wait()` é o mecanismo pelo qual o pai coleta esse status, permitindo que o kernel remova definitivamente a entrada do filho da tabela de processos. Sem o `wait()`, os processos filhos encerrados permanecem como zumbis indefinidamente, consumindo entradas na tabela de processos do sistema. Se muitos processos zumbis se acumularem, o sistema pode esgotar o número máximo de processos permitidos, impedindo a criação de novos processos.

7 Experimento 6: A Primeira Thread (Simples)

Este experimento demonstra a criação de uma thread simples utilizando a biblioteca Pthreads. O programa cria uma thread que imprime mensagens periodicamente, enquanto a thread principal aguarda com `pthread_join()`.

Utilizou-se o comando `ps -eL | grep exp01` para visualizar as threads do processo, onde a flag `-L` exibe os LWPs (*Light Weight Processes*), que correspondem às threads no Linux. Também foi utilizado o comando `lsuf -p [PID]` para listar os arquivos abertos pelo processo.

Na saída do `ps -eL`, é possível observar que o processo possui dois LWPs (threads): a thread principal e a thread criada por `pthread_create()`. Ambas compartilham o mesmo PID, mas possuem LWPs distintos, confirmando que threads existem dentro do mesmo espaço de endereçamento do processo.

Na saída do `lsuf`, observam-se os descritores de arquivo padrão (`stdin`, `stdout`, `stderr`) e os arquivos de bibliotecas compartilhadas carregadas pelo processo, como a `libpthread.so`.

```

=== ps -eL | grep ex6 ===
  40010  40010 ?      00:00:00 ex6
  40010  40012 ?      00:00:00 ex6

=== lsof -p 40010 ===
COMMAND  PID  USER  FD   TYPE DEVICE SIZE/OFF  NODE NAME
ex6      40010 bilic  cwd   DIR   8,48    4096 243896 /home/bilic/Engenharia-de-Telecomunicacoes/5th-semester/SOP/Prova-1/Aula-11/Lab-05
ex6      40010 bilic  rtd   DIR   8,48    4096     2 /
ex6      40010 bilic  txt   REG   8,48   16232 273797 /home/bilic/Engenharia-de-Telecomunicacoes/5th-semester/SOP/Prova-1/Aula-11/Lab-05/ex6
ex6      40010 bilic  mem   REG   8,48  2125328 12801 /usr/lib/x86_64-linux-gnu/libc.so.6
ex6      40010 bilic  mem   REG   8,48  236616 12598 /usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
ex6      40010 bilic  0r    CHR   1,3      0t0     4 /dev/null
ex6      40010 bilic  1w    REG   8,48    125 272958 /tmp/claude-1000/-home-bilic-Engenharia-de-Telecomunicacoes/9bf3886d-172b-4225-ae13-6bbafd154c61/tasks/bo4pdng6x.output
ex6      40010 bilic  2w    REG   8,48    125 272958 /tmp/claude-1000/-home-bilic-Engenharia-de-Telecomunicacoes/9bf3886d-172b-4225-ae13-6bbafd154c61/tasks/bo4pdng6x.output

=== ls -l /proc/40010/fd ===
total 0
lr-x----- 1 bilic bilic 64 Apr  6 15:44 0 -> /dev/null
l-wx----- 1 bilic bilic 64 Apr  6 15:44 1 -> /tmp/claude-1000/-home-bilic-Engenharia-de-Telecomunicacoes/9bf3886d-172b-4225-ae13-6bbafd154c61/tasks/bo4pdng6x.output
l-wx----- 1 bilic bilic 64 Apr  6 15:44 2 -> /tmp/claude-1000/-home-bilic-Engenharia-de-Telecomunicacoes/9bf3886d-172b-4225-ae13-6bbafd154c61/tasks/bo4pdng6x.output
[rerun: b7]

```

Figura 8: Saída dos comandos `ps -eL | grep exp01` e `lsof -p [PID]`

8 Experimento 7: O Contador de Threads

Neste experimento, foram criadas 4 threads com tempos de vida distintos (30s, 60s, 90s e 120s). O objetivo é observar o comportamento das threads ao longo do tempo utilizando o `htop`.

8.1 Respostas

1. Descreva o que acontece no `htop` a cada intervalo de 30 segundos. As linhas desaparecem ou mudam de cor?

A cada intervalo de 30 segundos, uma thread finaliza e sua linha desaparece do `htop`:

- Aos 30s: a Thread 1 encerra e sua linha desaparece, restando 4 linhas (processo pai + 3 threads).
- Aos 60s: a Thread 2 encerra, restando 3 linhas.
- Aos 90s: a Thread 3 encerra, restando 2 linhas.
- Aos 120s: a Thread 4 encerra, e logo em seguida o processo principal também finaliza.

2. No `htop`, as threads possuem o mesmo PID (Process ID) do processo pai?

Sim. No `htop`, todas as threads possuem o mesmo PID do processo pai. Isso ocorre porque threads compartilham o mesmo espaço de endereçamento e pertencem ao mesmo processo. O que as diferencia é o TID (*Thread ID*) ou LWP, que é um identificador único atribuído pelo kernel a cada thread.

3. Por que o processo principal (pai) só termina após a última thread (de 120s) finalizar?

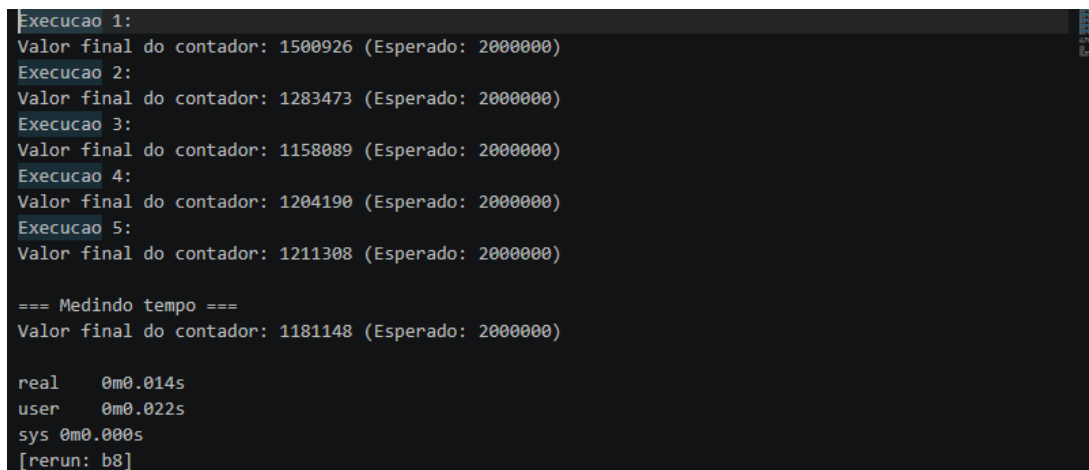
Porque o programa utiliza `pthread_join()` para cada uma das 4 threads. A função `pthread_join()` é bloqueante: ela faz com que a thread chamadora (neste caso, a thread principal) aguarde até que a thread especificada termine. Como o `join` é chamado sequencialmente para todas as threads, o processo principal só continua após a última thread (120s) encerrar.

4. Se você fechar o programa com Ctrl+C, o que acontece com as threads que ainda estavam rodando?

Ao pressionar Ctrl+C, o sinal SIGINT é enviado para o processo inteiro. Como todas as threads compartilham o mesmo processo, todas são encerradas imediatamente junto com o processo principal. Diferentemente de processos filhos criados com `fork()`, as threads não possuem existência independente do processo pai — quando o processo morre, todas as suas threads morrem junto.

9 Experimento 8: Concorrência e Race Condition

Neste experimento, duas threads incrementam simultaneamente uma variável global `contador` 1 milhão de vezes cada. O valor esperado ao final seria 2.000.000, mas devido à *race condition*, o valor obtido é frequentemente menor.



```
Execucao 1:
Valor final do contador: 1500926 (Esperado: 2000000)
Execucao 2:
Valor final do contador: 1283473 (Esperado: 2000000)
Execucao 3:
Valor final do contador: 1158089 (Esperado: 2000000)
Execucao 4:
Valor final do contador: 1204190 (Esperado: 2000000)
Execucao 5:
Valor final do contador: 1211308 (Esperado: 2000000)

=== Medindo tempo ===
Valor final do contador: 1181148 (Esperado: 2000000)

real    0m0.014s
user    0m0.022s
sys 0m0.000s
[rerun: b8]
```

Figura 9: Múltiplas execuções mostrando valores inconsistentes do contador

O resultado muda? Por quê?

Sim, o resultado muda a cada execução. Isso ocorre devido a uma *race condition* (condição de corrida). A operação `contador++` não é atômica — ela é composta por três etapas: (1) ler o valor atual de `contador` da memória para um registrador, (2) incrementar o valor no registrador e (3) escrever o novo valor de volta na memória.

Quando duas threads executam essas etapas simultaneamente, pode ocorrer a seguinte situação:

1. Thread A lê `contador` = 100
2. Thread B lê `contador` = 100 (antes de A escrever)
3. Thread A escreve `contador` = 101
4. Thread B escreve `contador` = 101 (sobrescrevendo, perdendo um incremento)

Esse problema é chamado de *race condition* porque o resultado depende da ordem (“corrida”) em que as threads acessam a região crítica. Para resolver, seria necessário utilizar mecanismos de exclusão mútua, como `pthread_mutex_lock()` e `pthread_mutex_unlock()`, garantindo que apenas uma thread acesse a variável por vez.

O comando `time ./programa` mostra que o tempo de execução é bastante curto (na ordem de milissegundos), o que confirma que ambas as threads executam em paralelo nos núcleos do processador.

10 Experimento 9: Monitoramento de Longa Duração

Este experimento cria três threads com comportamentos distintos: duas threads CPU-intensivas (que executam cálculos matemáticos em laço infinito) e uma thread de I/O (que imprime mensagens periodicamente com `sleep(1)`).

No `htop`, é possível observar que as duas threads CPU-intensivas consomem aproximadamente 100% de um núcleo cada, enquanto a thread de I/O permanece com uso de CPU próximo de 0%, pois passa a maior parte do tempo dormindo (`sleep`).

O comando `renice` foi utilizado para alterar a prioridade (*nice value*) do processo enquanto ele estava em execução. Por exemplo, `renice +10 -p [PID]` reduz a prioridade do processo, fazendo com que o escalonador do kernel dê menos tempo de CPU a ele em relação a outros processos. Isso demonstra como o sistema operacional permite o ajuste dinâmico de prioridades para gerenciar a alocação de recursos.

11 Experimento 10: Simulação de Servidor Web

Neste experimento, simula-se um servidor web com 5 threads *workers*, cada uma aguardando e processando “requisições” simuladas com tempos aleatórios.

Para colocar o servidor em background, utilizou-se `Ctrl+Z` (que envia o sinal `SIGTSTP`, suspendendo o processo) seguido do comando `bg` (que retoma a execução do processo em segundo plano).

Em seguida, utilizou-se `kill -9 [PID]` para enviar o sinal `SIGKILL` ao processo, encerrando-o forçadamente. Este sinal não pode ser capturado ou ignorado pelo processo, garantindo sua terminação imediata.

```
=== Servidor rodando em background. PID: 40631 ===
=== Usando kill -9 para derrubar o servidor ===
/bin/bash: line 16: 40631 Killed                  ./ex10
=== Servidor encerrado ===
[erun: b9]
```

Figura 10: Execução do servidor em background e encerramento com `kill -9`

Quando o `kill -9` é executado, o processo inteiro é encerrado, incluindo todas as 5 threads *workers*. Como as threads compartilham o mesmo espaço de processo, não é possível matar o processo sem encerrar todas as suas threads. Esse comportamento difere de processos filhos criados com `fork()`, que possuem existência independente e continuariam em execução mesmo que o processo pai fosse encerrado.

12 Desafio Final: Análise Teórica de Modelos de Threads

Nesta seção, analisa-se o comportamento do código proposto sob dois modelos de mapeamento de threads: **N:1** (Muitos para Um) e **1:1** (Um para Um).

12.1 A) Diagramas de Execução

Modelo 1:1 (Um para Um):

No modelo 1:1, cada thread de usuário é mapeada diretamente para uma thread do kernel. Assim, quando uma thread executa `sleep()`, apenas ela é bloqueada, e as demais continuam executando.

```

Tempo:  0s          1s          10s          11s          21s
        |-----|-----|-----|-----|-----|
main:   [cria tA, tB] [sleep(1)] [join tA] [join tB] [sleep(1)] [cria tC] [join tC]
tA:     [----sleep(10)----] [print] [exit]
tB:     [----sleep(10)----] [print] [exit]
tC:                                           [--sleep(10)--] [print] [exit]

```

Neste modelo, tA e tB executam em paralelo. Ambas iniciam o `sleep(10)` quase simultaneamente e terminam aos 10s. Após o `join` de ambas e o `sleep(1)` do main, tC é criada e roda por mais 10s.

Modelo N:1 (Muitos para Um):

No modelo N:1, todas as threads de usuário são mapeadas para uma única thread do kernel. Uma syscall bloqueante (`sleep`) bloqueia o processo inteiro.

```

Tempo:  0s          10s          20s          21s          31s
        |-----|-----|-----|-----|-----|
main:   [cria tA,tB]                               ...espera...
tA:     [---sleep(10)---] [print] [exit]
tB:                               [---sleep(10)---] [print] [exit]
main:                               [sleep(1)] [cria tC]
tC:                                           [---sleep(10)---] [print] [exit]

```

Neste modelo, quando tA executa `sleep(10)`, o processo inteiro é bloqueado por 10 segundos. Somente após tA terminar, tB pode executar seu `sleep(10)`, bloqueando novamente por mais 10s. O mesmo ocorre com tC.

12.2 B) Durações Aproximadas

- **Modelo 1:1:** Tempo total ≈ 22 segundos. As threads tA e tB executam em paralelo (10s), seguidas de `sleep(1)` do main (1s) e tC (10s), mais o `sleep(1)` do main antes do `join` de tC, totalizando aproximadamente $10 + 1 + 1 + 10 \approx 22$ s.
- **Modelo N:1:** Tempo total ≈ 32 segundos. As threads executam sequencialmente: tA (10s) + tB (10s) + `sleep(1)` do main + tC (10s) + `sleep(1)` do main, totalizando aproximadamente $10 + 10 + 1 + 10 + 1 \approx 32$ s.

12.3 C) Saída do Programa

A saída do programa é a mesma em ambos os modelos:

```

x: 1, y: 1
x: 1, y: 2
x: 1, y: 3

```

Justificativa:

- A variável `x` é **local** à função `threadBody()`, portanto cada thread possui sua própria cópia de `x`. Em cada chamada, `x` é inicializada com 0 e incrementada para 1. Por isso, `x` sempre vale 1 na saída.
- A variável `y` é **global**, compartilhada entre todas as threads. Cada thread incrementa `y` com `++y`. Como as threads executam sequencialmente em ambos os modelos (no N:1 por bloqueio, no 1:1 porque `tA` e `tB` terminam antes de `tC` começar, e o `join` garante a ordem), os incrementos ocorrem de forma ordenada: a primeira thread imprime `y=1`, a segunda `y=2` e a terceira `y=3`.

A saída não difere entre os modelos porque, embora a ordem de execução temporal seja diferente, o efeito sobre as variáveis é o mesmo: cada thread incrementa `y` uma vez, e o mecanismo de `join` garante que o `main` aguarde o término de cada thread antes de prosseguir.